

DBMS – SELLER AND INVENTORY MANAGEMENT

Project Title: IKEA E-Commerce Database Management System

1. Database Design

1.1 Seller Consideration

The proposed IKEA E-Commerce Database represents IKEA as a direct retailer rather than a multi-vendor marketplace. Therefore, IKEA itself acts as the seller in the system and a separate Seller table is not required. The Supplier table is maintained separately to represent suppliers who provide products to IKEA.

1.2 Product Table

Attribute	Key	Description
Product_ID	Primary Key	Unique identification of the product
Product_Name	—	Name of the product
Description	—	Description of the product
Price	—	Price of the product
Brand	—	Brand of the product
Weight	—	Weight of the product
Color	—	Colour of the product
Material	—	Material used for the product
Category_ID	Foreign Key	References Category table
Supplier_ID	Foreign Key	References Supplier table

1.3 Inventory Table

Attribute	Key	Description
Inventory_ID	Primary Key	Unique identification of inventory record
Product_ID	Foreign Key	References Product table
Warehouse_ID	Foreign Key	References Warehouse table
Stock_Quantity	—	Quantity of product currently available

The Inventory table maintains the stock information of IKEA products stored in different warehouses. Each inventory record identifies a product, its warehouse location and the available stock quantity.

2. Primary and Foreign Key Relationship

This corresponds to the reference's Section 2, where it explains the Seller → Inventory and Product → Inventory relationships.

2.1 Product → Inventory

Word document content

- Product.Product_ID → Primary Key
- Inventory.Inventory_ID → Primary Key
- Inventory.Product_ID → Foreign Key referencing Product.Product_ID
- Relationship: **One-to-Many (1:M)**

One product can have multiple inventory records, particularly when the same product is stored in different warehouses.

2.2 Warehouse → Inventory

- Warehouse.Warehouse_ID → Primary Key
- Inventory.Warehouse_ID → Foreign Key referencing Warehouse.Warehouse_ID

- Relationship: **One-to-Many (1:M)**

One warehouse can contain inventory records for multiple products.

2.3 Supplier → Product

- Supplier.Supplier_ID → Primary Key
- Product.Supplier_ID → Foreign Key referencing Supplier.Supplier_ID
- Relationship: **One-to-Many (1:M)**

One supplier can provide multiple products.

Relationship diagram



3. Table Creation

3.1 Create Seller Table

```
SQL> CREATE TABLE Seller (  
 2     Seller_ID NUMBER PRIMARY KEY,  
 3     Seller_Name VARCHAR2(100) NOT NULL,  
 4     Contact_Number VARCHAR2(15),  
 5     Email VARCHAR2(100),  
 6     Address VARCHAR2(200)  
 7 );
```

Table created.

The Seller table is created with Seller ID as the primary key. It stores seller details such as seller name, contact number, email, and address.

```
SQL> DESC Seller;  
Name                               Null?    Type  
-----  
SELLER_ID                          NOT NULL NUMBER  
SELLER_NAME                        NOT NULL VARCHAR2(100)  
CONTACT_NUMBER                     VARCHAR2(15)  
EMAIL                              VARCHAR2(100)  
ADDRESS                            VARCHAR2(200)
```

3.2 Create Inventory Table

```
SQL> CREATE TABLE Inventory (  
 2     Inventory_ID NUMBER PRIMARY KEY,  
 3     Product_ID NUMBER,  
 4     Warehouse_ID NUMBER,  
 5     Stock_Quantity NUMBER  
 6 );
```

Table created.

```
-----
PRICE
-----
BRAND
-----
WEIGHT COLOR
-----
MATERIAL
-----
CATEGORY_ID SUPPLIER_ID
-----
15 Brown

PRODUCT_ID
-----
PRODUCT_NAME
-----
DESCRIPTION
-----
PRICE
-----
BRAND
-----
WEIGHT COLOR
-----
MATERIAL
-----
CATEGORY_ID SUPPLIER_ID
-----
Wood

PRODUCT_ID
-----
PRODUCT_NAME
-----
DESCRIPTION
-----
PRICE
-----
BRAND
-----
WEIGHT COLOR
-----
MATERIAL
-----
CATEGORY_ID SUPPLIER_ID
-----
1 4

PRODUCT_ID
-----
PRODUCT_NAME
-----
DESCRIPTION
-----
PRICE
-----
BRAND
-----
WEIGHT COLOR
```

4.SELLER DATA INSERTION

```
SQL> INSERT INTO Seller VALUES
  2 (3, 'IKEA Hyderabad', '1800-419-4532', 'hyderabad@ikea.in', 'Hyderabad, Telangana');

1 row created.

SQL>
SQL> INSERT INTO Seller VALUES
  2 (4, 'IKEA Mumbai', '1800-419-4532', 'mumbai@ikea.in', 'Mumbai, Maharashtra');

1 row created.

SQL>
SQL> INSERT INTO Seller VALUES
  2 (5, 'IKEA Pune', '1800-419-4532', 'pune@ikea.in', 'Pune, Maharashtra');

1 row created.

SQL>
SQL> INSERT INTO Seller VALUES
  2 (6, 'IKEA Delhi', '1800-419-4532', 'delhi@ikea.in', 'New Delhi');

1 row created.

SQL>
SQL> INSERT INTO Seller VALUES
  2 (7, 'IKEA Noida', '1800-419-4532', 'noida@ikea.in', 'Noida, Uttar Pradesh');

1 row created.

SQL>
SQL> INSERT INTO Seller VALUES
  2 (8, 'IKEA Kolkata', '1800-419-4532', 'kolkata@ikea.in', 'Kolkata, West Bengal');

1 row created.

SQL>
SQL> INSERT INTO Seller VALUES
  2 (9, 'IKEA Ahmedabad', '1800-419-4532', 'ahmedabad@ikea.in', 'Ahmedabad, Gujarat');

1 row created.

SQL>
SQL> INSERT INTO Seller VALUES
  2 (10, 'IKEA Bengaluru', '1800-419-4532', 'bengaluru@ikea.in', 'Bengaluru, Karnataka');

1 row created.
```

5.INVENTORY DATA INSERTION

```
SQL> INSERT INTO Inventory VALUES (101, 101, 1, 25);

1 row created.

SQL> INSERT INTO Inventory VALUES (102, 102, 2, 18);

1 row created.

SQL> INSERT INTO Inventory VALUES (103, 103, 3, 0);

1 row created.

SQL> INSERT INTO Inventory VALUES (104, 104, 4, 32);

1 row created.

SQL> INSERT INTO Inventory VALUES (105, 105, 5, 15);

1 row created.

SQL> INSERT INTO Inventory VALUES (106, 106, 6, 0);

1 row created.

SQL> INSERT INTO Inventory VALUES (107, 107, 7, 40);

1 row created.

SQL> INSERT INTO Inventory VALUES (108, 108, 8, 12);

1 row created.

SQL> INSERT INTO Inventory VALUES (109, 109, 9, 27);

1 row created.

SQL> INSERT INTO Inventory VALUES (110, 110, 10, 0);

1 row created.

SQL>
SQL> COMMIT;

Commit complete.
```

6.SEKKER DATA DISPLAY

```
SQL> SELECT * FROM Seller;

 SELLER_ID
-----
 SELLER_NAME
-----
CONTACT_NUMBER
-----
EMAIL
-----
ADDRESS
-----
1
IKEA India
1880-419-4532

 SELLER_ID
-----
 SELLER_NAME
-----
CONTACT_NUMBER
-----
EMAIL
-----
ADDRESS
-----
support@ikea.in
Bengaluru, Karnataka

 SELLER_ID
-----
 SELLER_NAME
-----
CONTACT_NUMBER
-----
EMAIL
-----
ADDRESS
-----
2
IKEA Chennai
1880-419-4532

 SELLER_ID
-----
 SELLER_NAME
-----
CONTACT_NUMBER
-----
EMAIL
-----
ADDRESS
-----
chennai@ikea.in
Chennai, Tamil Nadu

 SELLER_ID
```

7. INVENTORY DATA DISPLAY

```
SQL>
SQL> SELECT * FROM Inventory;

INVENTORY_ID PRODUCT_ID WAREHOUSE_ID STOCK_QUANTITY
-----
          101          101             1             25
          102          102             2             18
          103          103             3              8
          104          104             4             32
          105          105             5             15
          106          106             6              8
          107          107             7             48
          108          108             8             12
          109          109             9             27
          110          110            10              8

10 rows selected.

SQL> 
```

8. Seller Product Information

```
SQL> SELECT
  2   s.Seller Name,
  3   p.Product_ID,
  4   p.Product Name,
  5   p.Price,
  6   i.Stock Quantity
  7 FROM Seller s
  8 CROSS JOIN Product p
  9 JOIN Inventory i
10 ON p.Product_ID = i.Product_ID
11 WHERE s.Seller_ID = 1;

SELLER NAME
-----
PRODUCT_ID
-----
PRODUCT NAME
-----
PRICE STOCK QUANTITY
-----
DEFA India
901
KALLAX Shelf Unit
4000 25

SELLER NAME
-----
PRODUCT_ID
-----
PRODUCT NAME
-----
PRICE STOCK QUANTITY
-----
DEFA India
902
KALLA Bed Frame
12000 18

SELLER NAME
-----
PRODUCT_ID
-----
PRODUCT NAME
-----
PRICE STOCK QUANTITY
-----
DEFA India
903
LACK Coffee Table
2400 8

SELLER NAME
-----
PRODUCT_ID
-----
PRODUCT NAME
-----
PRICE STOCK QUANTITY
-----
DEFA India
904
FORNS Armchair
6000 32

SELLER NAME
-----
PRODUCT_ID
-----
PRODUCT NAME
-----
```



```

-----
PRICE STOCK QUANTITY
-----
DATA India
105
ROCKE Desk
5999          15

SELLER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE STOCK QUANTITY
-----
DATA India
106
MOOLF Dining Chair
3999          8

SELLER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE STOCK QUANTITY
-----
DATA India
107
MCO Table Lamp
1999          40

SELLER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE STOCK QUANTITY
-----
DATA India
108
MULTI Clothes Rack
1999          12

SELLER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE STOCK QUANTITY
-----
DATA India
109
APPLARD Outdoor Table
8999          27

SELLER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
PRICE STOCK QUANTITY
-----
DATA India
110
REWA Artificial Plant
799          8

8 rows selected.

SQL>

```

Seller product information was retrieved by combining the Seller, Product, and Inventory tables. The query displays the seller name, product details, price, and current stock quantity.

9. Product Availability

9.1 Available Products

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     i.Stock_Quantity,
  5     'Available' AS Availability_Status
  6 FROM Product p
  7 JOIN Inventory i
  8 ON p.Product_ID = i.Product_ID
  9 WHERE i.Stock_Quantity > 0;

PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY AVAILABL
-----
101
KALLAX Shelf Unit
25 Available

102
KALLAX Bed Frame
18 Available

PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY AVAILABL
-----
104
KIVIKG Armchair
32 Available

105
POCKE Desk

PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY AVAILABL
-----
106
FACKA Table Lamp
40 Available

108

PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY AVAILABL
-----
109
MULIG Clothes Rack
12 Available

100
AVFARO Outdoor Table
27 Available

7 rows selected.
```

Products with a stock quantity greater than zero are classified as available. The query retrieves the product ID, product name, stock quantity, and availability status.

9.2 Unavailable Products

Products with zero stock quantity are classified as unavailable. This allows the inventory system to identify products that require restocking.

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     i.Stock_Quantity,
  5     'Unavailable' AS Availability_Status
  6 FROM Product p
  7 JOIN Inventory i
  8 ON p.Product_ID = i.Product_ID
  9 WHERE i.Stock_Quantity = 0;
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY AVAILABILIT
-----
      103
LACK Coffee Table
           0 Unavailable

      106
INGOLF Dining Chair
           0 Unavailable

PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY AVAILABILIT
-----

      110
FEJKA Artificial Plant
           0 Unavailable
```

10.Inventory Update

The stock quantity of an inventory record was updated using the UPDATE statement. The transaction was committed successfully and the updated record was verified using a SELECT statement.

```
SQL> UPDATE Inventory
  2 SET Stock_Quantity = 20
  3 WHERE Inventory_ID = 103;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT *
  2 FROM Inventory
  3 WHERE Inventory_ID = 103;

INVENTORY_ID PRODUCT_ID WAREHOUSE_ID STOCK_QUANTITY
-----
      103      103           3           20
```

11. Inventory Status Reports

11.1 Complete Inventory Report

The complete inventory report combines product, warehouse, stock quantity, and availability information. It provides an overall view of the current inventory status.

```
SQL> SELECT
  2   i.Inventory_ID,
  3   p.Product_ID,
  4   p.Product_Name,
  5   w.Warehouse_ID,
  6   w.Warehouse_Name,
  7   i.Stock_Quantity,
  8   CASE
  9     WHEN i.Stock_Quantity > 0 THEN 'Available'
 10    ELSE 'Unavailable'
 11  END AS Availability_Status
 12 FROM Inventory i
 13 JOIN Product p
 14 ON i.Product_ID = p.Product_ID
 15 JOIN Warehouse w
 16 ON i.Warehouse_ID = w.Warehouse_ID
 17 ORDER BY i.Inventory_ID;
```

INVENTORY_ID PRODUCT_ID

PRODUCT_NAME

WAREHOUSE_ID

WAREHOUSE NAME

STOCK_QUANTITY AVAILABILITY

101 101
FALLAX Shelf Unit
1

INVENTORY_ID PRODUCT_ID

PRODUCT NAME

WAREHOUSE_ID

WAREHOUSE NAME

STOCK_QUANTITY AVAILABILITY

Bengaluru Warehouse
25 Available

INVENTORY_ID PRODUCT_ID

PRODUCT NAME

WAREHOUSE_ID

WAREHOUSE NAME

STOCK_QUANTITY AVAILABILITY

102 102
PALM Bed Frame
2

INVENTORY_ID PRODUCT_ID

PRODUCT NAME

WAREHOUSE_ID

WAREHOUSE NAME

STOCK_QUANTITY AVAILABILITY

Chennai Warehouse
18 Available

INVENTORY_ID PRODUCT_ID

PRODUCT NAME

WAREHOUSE_ID

WAREHOUSE NAME

STOCK_QUANTITY AVAILABILITY

103 103

103 103
LACK Coffee Table
3

INVENTORY_ID PRODUCT_ID

PRODUCT_NAME

WAREHOUSE_ID

WAREHOUSE_NAME

STOCK QUANTITY AVAILABILITY

Hyderabad Warehouse
20 Available

INVENTORY_ID PRODUCT_ID

PRODUCT_NAME

WAREHOUSE_ID

WAREHOUSE_NAME

STOCK QUANTITY AVAILABILITY

104 104
FUNK Armchair
4

INVENTORY_ID PRODUCT_ID

PRODUCT_NAME

WAREHOUSE_ID

WAREHOUSE_NAME

STOCK QUANTITY AVAILABILITY

Mumbai Warehouse
32 Available

INVENTORY_ID PRODUCT_ID

PRODUCT_NAME

WAREHOUSE_ID

WAREHOUSE_NAME

STOCK QUANTITY AVAILABILITY

105 105
PECKE Desk
5

INVENTORY_ID PRODUCT_ID

PRODUCT_NAME

WAREHOUSE_ID

WAREHOUSE_NAME

STOCK QUANTITY AVAILABILITY

Pune Warehouse
15 Available

INVENTORY_ID PRODUCT_ID

PRODUCT_NAME

WAREHOUSE_ID

WAREHOUSE_NAME

STOCK QUANTITY AVAILABILITY

106 106

```
INVENTORY_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
WAREHOUSE_ID
-----
WAREHOUSE_NAME
-----
STOCK QUANTITY AVAILABILITY
-----
Delhi Warehouse
      0 Unavailable
```

```
INVENTORY_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
WAREHOUSE_ID
-----
WAREHOUSE_NAME
-----
STOCK QUANTITY AVAILABILITY
-----
      187      187
FACO Table Lamp
      7
```

```
INVENTORY_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
WAREHOUSE_ID
-----
WAREHOUSE_NAME
-----
STOCK QUANTITY AVAILABILITY
-----
Noida Warehouse
      40 Available
```

```
INVENTORY_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
WAREHOUSE_ID
-----
WAREHOUSE_NAME
-----
STOCK QUANTITY AVAILABILITY
-----
      188      188
MULTI Clothes Rack
      8
```

```
INVENTORY_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
WAREHOUSE_ID
-----
WAREHOUSE_NAME
-----
STOCK QUANTITY AVAILABILITY
-----
Kolkata Warehouse
      12 Available
```

```
INVENTORY_ID PRODUCT_ID
-----
PRODUCT_NAME
-----
WAREHOUSE_ID
-----
WAREHOUSE_NAME
-----
STOCK QUANTITY AVAILABILITY
-----
      189      189
APPLAFD Outdoor Table
      9
```

11.2 Total Stock by Supplier

The total stock available for each supplier was calculated using the SUM aggregate function. The results are grouped according to the supplier associated with each product.

```
sql> select
  2  s.supplier_id
  3  s.supplier_name
  4  sum(i.stock_quantity) as total_stock
  5  from supplier s
  6  join product p
  7  on s.supplier_id = p.supplier_id
  8  join inventory i
  9  on p.product_id = i.product_id
 10 group by
 11  s.supplier_id,
 12  s.supplier_name
 13 order by s.supplier_id;

supplier_id
-----
supplier_name
-----
total_stock
-----
1
BIO Supplier 1
25
2
BIO Supplier 2
25
supplier_id
-----
supplier_name
-----
total_stock
-----
3
BIO Supplier 3
25
4
BIO Supplier 4
supplier_id
-----
supplier_name
-----
total_stock
-----
10
5
BIO Supplier 5
25
6
supplier_id
-----
supplier_name
-----
total_stock
-----
BIO Supplier 6
0
7
BIO Supplier 7
0
supplier_id
-----
supplier_name
-----
total_stock
-----
8
BIO Supplier 8
15
9
BIO Supplier 9
25
supplier_id
-----
supplier_name
-----
total_stock
-----
20
BIO Supplier 20
0
20 rows selected.

sql> |
```

11.3 Products with Zero Stock

The report identifies products whose current stock quantity is zero. These products are currently unavailable and may require restocking.

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     i.Stock_Quantity
  5 FROM Product p
  6 JOIN Inventory i
  7 ON p.Product_ID = i.Product_ID
  8 WHERE i.Stock_Quantity = 0;
```

```
PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY
-----
      106
INGOLF Dining Chair
      0

      110
FEJKA Artificial Plant
      0

PRODUCT_ID
-----
PRODUCT_NAME
-----
STOCK_QUANTITY
-----
```

11.4 Available and Unavailable Product Count

The number of available and unavailable products was calculated using the COUNT aggregate function. Product availability is determined based on the current stock quantity.

```
SQL> SELECT
  2     CASE
  3         WHEN Stock_Quantity > 0 THEN 'Available'
  4         ELSE 'Unavailable'
  5     END AS Availability_Status,
  6     COUNT(*) AS Product_Count
  7 FROM Inventory
  8 GROUP BY
  9     CASE
 10         WHEN Stock_Quantity > 0 THEN 'Available'
 11         ELSE 'Unavailable'
 12     END;
```

```
AVAILABILIT PRODUCT_COUNT
-----
Available      8
Unavailable     2
```